Artificial Intelligence Approaches to the Construction Aspect of Catan

Nicholas Rising *rising.n@northeastern.edu*
Jason Snyder *snyder.jas@northeastern.edu*
Siddarth Srinath *srinath.s@northeastern.edu*
Mohammed Ahnaf Tajwar *tajwar.m@northeastern.edu*

***Abstract*—This project tested four different implementations of an artificial intelligence model focused on construction actions in *Settlers of Catan*. Those approaches included a modified Monte-Carlo Tree Search, an implementation of genetic algorithms, Temporal Difference Learning, and Deep Q-Learning. Evaluating these methods by win rate, Deep Q-Learning and the genetic algorithm showed the best performance. Further work should expand on the remaining features of base-game *Catan*, and also its expansion sets.**
The GitHub repository for this project is available at https://github.com/nicorising/catan-ai.

1. Introduction

2025 marks the 30th anniversary of *Settlers of Catan* [1], a board game that has sold over 40 million copies as of 2022 [2]. *Catan* is won if a player earns a certain amount of victory points [3]. This is done with development cards, by building and upgrading settlements, or by completing achievements, such as the "longest road" [3]. These methods require a certain combination of resources, which are randomly allocated during each player's turn [3], making *Catan* heavily non-deterministic. Additionally, standard games of *Catan* are partially-observable, since development cards remain hidden until played [3]. Players are also able to trade with each other, move a robber to steal an opponent's resource, and exchange resources at harbor locations [3].

These complexities have made *Catan* an enticing target for the development of artificial intelligence models. Some attempts to do so include Monte-Carlo Tree Search (MCTS) [4] and various reinforcement-learning strategies [5].

In this project we focus solely on the construction and upgrading of settlements, as incorporating all of *Catan* into a model is outside the scope of one semester. Without development cards our environment is fully-observable. We tried four different approaches to playing Catan intelligently: MCTS, genetic algorithms, Temporal Difference Learning (TDL), and Deep Q-Learning (DQL).

2. Related Work

Computer science research on *Settlers of Catan* has explored a wide range of approaches. Pfeiffer [5] used Reinforcement Learning to learn strategic policies. Branca and Johansson [6] used multi-agent systems to simulate interactions among *Catan* bots. Szita et al. [4] implemented MCTS in a simplified *Catan*, removing components such as development cards. Afantenos et al. [7] developed a corpus of conversations among *Catan* players to understand negotiation behavior. Similarly, Guhe and Lascarides [8] analyzed persuasion tactics with gameplay data, interpreted through a theoretical framework.

Cuayáhuitl et al. [9] used DQL to manage strategic dialogue policies in negotiation scenarios, rather than directly performing trade actions. Xenou et al. [10] incorporate LSTM-based sequence modeling into Deep Reinforcement Learning frameworks for long-term strategic planning in board game environments. Gendre and Kaneko [11] showed that cross-dimensional neural network architecture might be superior to Reinforcement Learning approaches in two-player games of *Catan*.

JSettlers is a Java implementation of *Catan* that provides a custom game environment with support for human and AI players [12]. It uses rule-based bots that make trading, building, and resource management decisions with fixed heuristics. Though useful as a baseline for testing, these bots lack the ability to learn complex strategies.

QSettlers improves on JSettlers by using Deep Q-Networks (DQN) for trade negotiations [13]. Simple heuristic-based logic is replaced with a learned policy, enabling the *Catan* agent to make more informed trading decisions. However, QSettlers only addressed the trading aspect of *Catan* [13].

Our approach uses the Catanatron environment [14], a Python library for implementing *Catan* agents. Catanatron provides detailed state tracking and extensive logging of game states, agent actions, and rewards.

3. Methodology

3.1. Monte-Carlo Tree Search

Rather than having each node of our MCTS graph correspond to a specific game state, they map to possible actions from the current state. Therefore, parent nodes represent the most-recent action. With this representation, we avoid having to present all probabilistic outcomes from the current state as child nodes. Rather, at any given decision-point during the game, our MCTS agent evaluates all child nodes by performing their corresponding actions on a copy of the game, and then runs simulations for the rest of that game, in which actions are chosen randomly. After all child nodes

have had their actions simulated, a simple heuristic is used to evaluate the best choice of action. Our MCTS agent defaults to 5 game simulations for each action.

Action Evaluation. Each action was evaluated with the following heuristic:

$$\text{value} = w + (0.1 * (v \div s))$$

- w is the amount of wins for that action's simulations
- v is the average amount of victory points for this player in the simulations that they lost
- s is the amount of simulations

By weighing w much more than $v \div s$, we prioritize actions that lead to us winning the game. However, v still provides valuable information. Even if all simulated games are lost, prioritizing actions which lead to the most victory points increases the likelihood of choosing an action that gets us closer to a win.

3.2. Temporal Difference Learning

The Catan game environment poses a challenge for Reinforcement Learning due to its delayed rewards and branching decision space. TDL was selected for its ability to incrementally update state values after each action without requiring full episodes to conclude, allowing more frequent learning updates and faster convergence in complex sequential environments.

Unlike more advanced methods like DQN, our TDL uses a tabular implementation with a discrete state representation. This allows for a clearer interpretation of the learned value function. This restriction reduced the size of the action space and helped manage the size of the q-table maintained by the algorithm.

Learning Algorithm. The TDL update rule used was:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \right]$$

Where:

- $V(s_t)$ is the value of the current state
- α is the learning rate
- γ is the discount factor
- r_{t+1} is the immediate reward received

The reward function was designed to reflect both win/loss outcomes and progress toward victory. If the agent won the game, it received a positive reward equal to the number of victory points required to win. If it lost, it received a negative reward of the same magnitude. Otherwise, the reward reflected the agent's relative progress toward victory, computed as the difference between its current victory points and the number needed to win. This formulation encouraged both maximizing long-term gains and reaching the win condition quickly.

Our TDL used an ϵ-greedy exploration strategy, with the initial ϵ set to 1.0. ϵ had a decay rate of 0.8 per episode applied with multiplication. This value encouraged broad exploration early in training, and more deterministic behavior as learning progressed.

Training Regime. The agent was trained through self-play, with TDL agents competing against each other. This enabled the agent to improve by learning from its own decisions and outcomes without relying on hard-coded opponent strategies. Approximately 10,000 episodes were simulated, terminating when an agent reached 10 victory points or when a maximum number of turns was reached. Hyperparameters included a learning rate $\alpha = 0.1$ and a discount factor $\gamma = 0.99$.

Algorithm 1 Temporal Difference Training

for episode $\leftarrow 1$ to NUM_EPISODES **do**
  Initialize environment and get initial state s
  total_reward $\leftarrow 0$
  while game not over and turn $<$ MAX_TURNS **do**
    $A \leftarrow$ get playable actions for state s
    Remove trade-related actions from A
    if random number $< \epsilon$ **then**
      $a \leftarrow$ random action from A ▷ exploration
    else
      $a \leftarrow \arg\max_{a'} Q(s, a')$ ▷ greedy action
    Execute action a and observe reward r and next state s'
    total_reward $\leftarrow$ total_reward $+ r$
    $A' \leftarrow$ get playable actions for state s'
    Remove trade-related actions from A'
    $a' \leftarrow \arg\max_{a'} Q(s', a')$
    target $\leftarrow r + \gamma \cdot Q(s', a')$
    $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (\text{target} - Q(s, a))$
    $s \leftarrow s'$
  $\epsilon \leftarrow \epsilon \cdot$ decay_rate
  Log total_reward for the episode

3.3. Genetic Algorithms

Genetic algorithms (GA) enable an agent to improve decision-making by evolving strategy weights through evaluation self-play. GA models learn by playing games against themselves, improving their strategies over generations.

Designing feature representation for this model was challenging. The feature vector, which ultimately consisted of 14 dimensions, was not created all at once. Initially it was difficult to decide which aspects of *Catan* were relevant when evaluating construction actions. We began by adding basic components like action type and current victory points. Through experimentation and iterative refinement, we expanded the feature set to include nuanced elements, such as board position quality, resource scarcity, blocking potential, and longest road potential. This incremental approach helped ensure that each feature added value to the agent's decision-making process.

Each strategy, or individual in the GA population, is represented by a 14-dimensional weight vector. When deciding on a move, the model calculates a score for each legal action using a linear function:

$$f(a) = w_1 \cdot f_1 + w_2 \cdot f_2 + w_3 \cdot f_3 + \cdots + w_n \cdot f_n = \mathbf{w} \cdot \mathbf{f}(a)$$

where $\mathbf{w}$ is the weight vector and $\mathbf{f}(a)$ is the feature vector of action a. The action with the highest score is selected.

Training. The training process consists of three main stages: initialization, fitness evaluation, and the evolutionary loop.

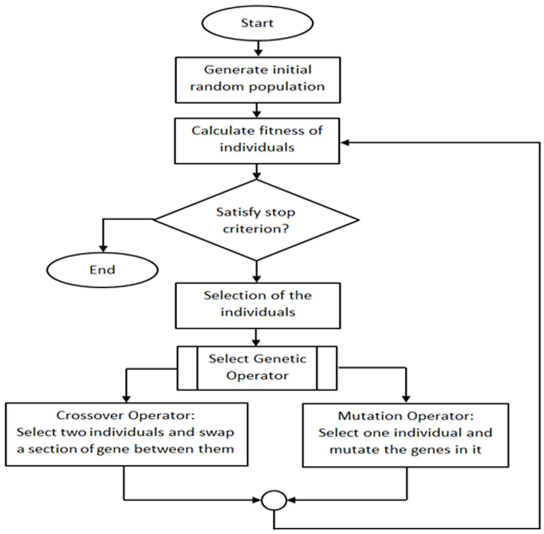


Figure 1: Flowchart of the Genetic Algorithm process [15].

Initialization. At the start, the population is initialized with random weight values in the range $[-1, 1]$. To guide early evolution, a simplified fitness estimate is assigned to each individual, rewarding weights that favor important construction features like settlements and cities, while penalizing high variance.

Fitness Evaluation. True fitness is measured through self-play. Each individual plays multiple full games against clones of itself using the same weights. Fitness is calculated using a weighted combination of win rate and average victory points, encouraging successful strategies, but which also perform well in games they do not win.

Evolution Process. The evolution process follows a $(\mu + \lambda)$ strategy. In each generation, three agents with different strategies play in a tournament. The strongest agent from each group is selected as a parent.

New agents are then created by combining strategies from two parents using two-point crossover, which occurs with a probability of 0.8. To encourage variety and prevent strategies from becoming too similar, each gene (or decision weight) has a 15% chance of being slightly altered through mutation. This mutation rate is adaptive: it starts high to promote exploration and gradually decreases to allow fine-tuning as stronger strategies emerge.

We used elitism to ensure that the best-performing agent from each generation, based on win rate and average victory points, is carried forward unchanged. This loop continues until a fixed number of generations is reached or a fitness threshold of 15.0 is achieved.

Algorithm 2 Genetic Algorithm Training

```
Initialize population P with N individuals
Evaluate simplified fitness, update best individual
for generation = 1 to G do
    for each ind in P do
        Play games against clone using ind's weights
        Compute fitness = win rate + avg. VP score
        Update best individual if applicable
    Select parents via tournament selection
    Apply crossover and mutation to generate new population
    Carry over best individual (elitism)
Save best weights
```

This GA agent was trained over 50 generations, with each individual evaluated through multiple self-play games. During training, the best-performing individual from each generation was logged, along with the corresponding maximum, minimum, and average fitness scores. The population's average fitness steadily improved across generations. In generation 1, the average fitness was approximately 0.70, with the best individual reaching a score of 4.0. By generation 25, the average fitness had increased to 2.6, and the best score reached 6.0. At generation 50, the average fitness rose to 2.7, with the top-performing individual achieving a fitness score of 10.0. The final trained model is defined by this best-performing weight vector (strategies), which is saved to a file and later loaded by the agent to make strategic decisions during actual gameplay in *Catan*.

3.4. Deep Q-Learning

The final model explored for application to the Catan environment uses DQN to approximate a Q-function for estimating future reward. The DQN agent receives the state of the game as a 228-element tensor encoding player victory points, resource cards in hand, the resource of each tile of the board (one hot encoded), and the status of each node and edge on the board (0 for nothing, 1 for settlement/road, 2 for city; positive for friend, negative for enemy). The DQN then outputs a 127-element reward estimate for building at each of the nodes and edges on the board, along with ending the turn. When taking an action, invalid moves are masked away and the action with the highest estimated reward is taken. Note that building on a node can constitute construction of a settlement or the upgrade of a settlement to a city, depending on whether the node is empty or contains a friendly settlement. The reward structure used during training associates each victory point with +1 reward, along with a +2 and -2 reward for winning and losing, respectively.

The network consists of five fully connected layers using the ReLU activation function, and implements dropout with $p = 0.1$ following each of the three hidden layers to deter overfitting. When training, two networks are initialized with random weights: the policy and target network, and the agent uses an ϵ-greedy policy to first take random actions and then gradually incorporate the policy network recommendation.

As games are played, state transitions are stored in a replay memory buffer. Following each game, this buffer is then sampled from to compute mean squared error loss between the predicted reward of the policy network and the more accurate future reward calculated as the sum of the true transition reward and the target network's prediction of the reward in the next state. The latter is also multiplied by the discount factor $\Gamma = 0.99$. Gradient descent is then performed on the policy network, and the target network has its weights linearly interpolated towards it by the "update rate," chosen empirically to be $\tau = 0.005$. This use of a target network that lags behind the policy network, only being updated in discrete steps has been shown to improve training outcomes, where a single policy network being both trained and used to predict future reward at the same time might result in divergence and/or oscillation as its weights change in the middle of training [16].

Algorithm 3 DQN Training Epoch

// Sample transitions from the memory replay buffer
$(s, a, next_s, r) \leftarrow memory.sample(BATCH_SIZE)$

// Get policy network prediction for each state-action pair
$r_est \leftarrow policy_network(s)$
for $idx \leftarrow 1...BATCH_SIZE$ **do**
    $r_est[idx] \leftarrow r_est[idx][a[idx]]$

// Determine the "true" reward with the target network
$r_tgt \leftarrow r$
$next_r_est \leftarrow target_network(next_s)$
for $idx \leftarrow 1...BATCH_SIZE$ **do**
    $r_tgt[idx] \leftarrow r_tgt[idx] + \Gamma \cdot next_r_est[idx].max()$

// Perform gradient descent on the policy network
$loss \leftarrow MeanSquareError(r_est, r_tgt)$
$policy_network.optimize(loss)$

// Lerp the target network weights towards the policy network weights
for $idx \leftarrow 1...LAYER_COUNT$ **do**
    $t_layer \leftarrow target_network.layers[idx]$
    $p_layer \leftarrow policy_network.layers[idx]$
    $t_layer \leftarrow (1 - \tau) \cdot t_layer + \tau \cdot p_layer$

4. Results

Analysis of models was performed against a random agent, choosing among the same actions that our intelligent agents were limited to (i.e., only construction). Each game was played on a board with tiles randomized in the same fashion as a standard game of over-the-board *Catan*. Due to constraints imposed by the Catanatron environment used for testing, a turn limit of 1,000 moves was enforced. Due to our focus on construction disallowing trading and development cards, some board states are extraordinarily hard to win on, and so drawn states that exceeded the turn limit were ignored.

MCTS had a win rate of 56% against the random agent. This indicates that our MCTS is not much of an improvement against a player that randomly chooses moves. A better-performing MCTS might actually account for every state as a node, use the tree to decide actions in simulations, and back-propagate every node. With more computing power, we could perform a greater number of simulations that would be more typical for MCTS implementations [4].

After training, the TDL agent consistently achieved win rates above 65% against a random agent, demonstrating an ability to learn long-term strategies. Our simplified version of *Catan* limited strategic depth, so TDL might have different results in more complicated games. Scaling tabular TDL is difficult, and methods like Deep Reinforcement Learning might improve its ability to handle full versions of *Catan*.

Using the optimized weights from training, the GA agent showed clear strategical improvements. When tested against a random agent, it achieved a 72% win rate. The GA agent consistently chose better settlement locations and made moves allowing it to gain more victory points over time. With additional training, we believe the GA agent could continue to refine its strategies and perform even better, though it has some limitations. Training is computationally expensive. With a population size of 30 and three evaluation games per individual, each generation requires around 90 full-game simulations, leading to long runtimes.

For the DQN agent, training was performed over 20,000 games against a random agent with a replay memory size of 100,000 transitions and a batch size of 10,000, where one epoch was computed following each game. This trained DQN agent displays a win-rate of roughly 73% over a random agent. The use of a neural network appears to successfully identify features and generalize to the *Catan* environment. However, it still struggles to learn an effective policy for initial settlement placement given a randomized board, likely due to the reward sparsity amid a high number of moves per game.

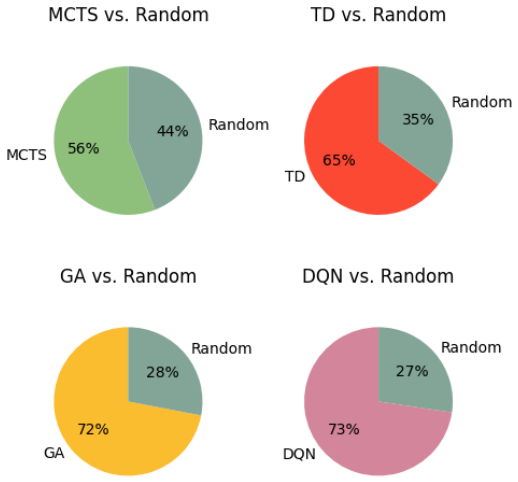


Figure 2: Model vs. random agent win percentage.

Beyond win rate, we also found the TDL and GA agents to be the fastest in per-move compute time. Deep Q-Learning runs a bit slower, and Monte-Carlo Tree Search

takes much longer than the other three due to its computationally expensive simulated rollouts of the game tree. Some of this performance hit may be a result of our particular *Catan* environment and its representation of game state, but fundamentally TD's recall time to determining the value of a state from its Q-table is understandably much faster than a Monte-Carlo method that must be re-applied each turn.

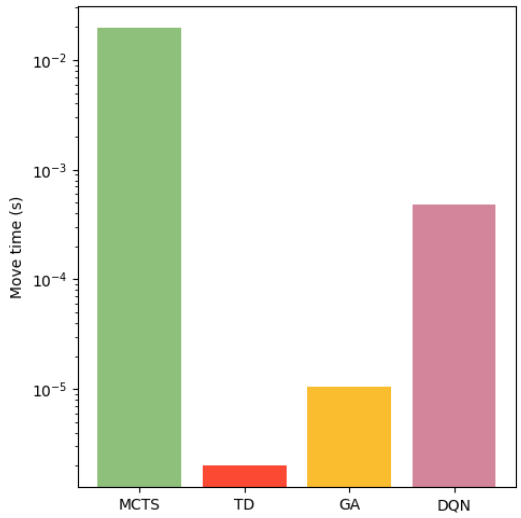


Figure 3: Model average turn time (seconds, logarithmic).

5. Conclusion

Overall, the DQN agent performs the best, followed by the GA agent, then TDL and MCTS, in that order. Our greatest challenge implementing *Catan* with Reinforcement Learning approaches is propagating reward over the game's duration. Arguably the most important move of *Catan* is the first turn, when players place their first settlement. This determines the resources they will have available to them at the start, which further enables them to build and upgrade settlements. To learn an adequate strategy for the opening, AI agents likely require prolonged training regiments to properly propagate reward from the relatively sparse victory points, or finely-tuned reward structures to account for this sparsity. Further research might explore this reward structure and identify ways in which it can be leveraged to teach intelligent policies faster.

For the Genetic Algorithm model, future improvements could include training for more generations with better hardware, fine-tuning key hyperparameters, expanding the feature vector to capture more strategic behaviors, and using a curriculum-based training approach to improve learning efficiency and generalization.

Specific to the DQN model, one area of particular interest is the application of convolutions to enhance feature-learning. Convolutional Neural Networks (CNN) have been applied to board games in the past, such as AlphaGo, which could play Go with great success [17]. *Catan* has a spatial element that would seem to encourage a CNN approach. Convolutions might help a DQN agent more rapidly understand the notion of controlling space and blocking opponents with roads. However, an obvious barrier to CNNs is the hexagonal structure of *Catan*. Still, hexagonal CNNs have been explored in recent research, and may provide a framework for native hexagonal kernels applied to the game of *Catan* [18].

6. Team Contributions

Nicholas designed the DQN agent, worked on visualizing games in the Catanatron environment, and contributed to the project proposal, presentation, and final report.

Jason implemented the MCTS agent, helped to design the presentation, coordinated meetings, edited all written work, and also wrote the abstract, introduction, and MCTS sections of the final project report.

Siddarth evaluated Catan environments, implemented the TDL agent, worked on the presentation, project proposal, and project report (section on Temporal Difference learning).

Ahnaf evaluated Catan environments, implemented the genetic algorithms agent, worked on the presentation, project proposal, and project report (wrote related work and genetic algorithms section).

References

[1] Catan GmbH, *CATAN's 6th edition will be released in 2025*, C, 2025. [Online]. Available: https://www.catan.com/catan-fans/news/catans-6th-edition-will-be-released-2025.

[2] Catan GmbH, *About Us*. [Online]. Available: https://www.catan.com/about-us.

[3] Catan GmbH, *CATAN - Game Rules Almanac (3-4 Players)*, www.catan.com. [Online]. Available: https://www.catan.com/sites/default/files/2021-06/catan_base_rules_2020_200707.pdf.

[4] *Monte-Carlo Tree Search in Settlers of Catan*, Advances in Computer Games, May 2009.

[5] M. Pfeiffer, "Reinforcement learning of strategies for settlers of catan," Ph.D. dissertation. [Online]. Available: https://www.researchgate.net/profile/Michael-Pfeiffer-11/publication/228728063_Reinforcement_learning_of_strategies_for_Settlers_of_Catan / links / 546ddfb80cf2193b94c5da66 / Reinforcement - learning-of-strategies-for-Settlers-of-Catan.pdf.

[6] L. Branca and S. J. Johansson, *Using multi-agent system technologies in settlers of catan bots*, DIVA, 2018. [Online]. Available: https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A837034&dswid=4844 (visited on 04/18/2025).

[7] S. Afantenos, N. Asher, F. Benamara, A. Cadilhac, and L. Vieu, *Developing a corpus of strategic conversation in the settlers of catan*, Sep. 2012. [Online]. Available: https : / / www . researchgate . net / publication / 281885354_Developing_a_corpus_of_strategic_conversation_in_The_Settlers_of_Catan.

[8] *The effectiveness of persuasion in The Settlers of Catan*, IEEE, Aug. 2014.

[9] H. Cuayáhuitl, S. Keizer, and O. Lemon, "Strategic dialogue management via deep reinforcement learning," vol. arXiv, Nov. 2015. [Online]. Available: https : / / www . researchgate . net / publication / 285271235_Strategic_Dialogue_Management_via_Deep_Reinforcement_Learning.

[10] K. Xenou, G. Chalkiadakis, and S. Afantenos, "Deep reinforcement learning in strategic board game environments," *Multi-Agent Systems*, pp. 233–248, 2019. DOI: 10 . 1007 / 978 - 3 - 030 - 14174 - 5_16. [Online]. Available: https : / / hal . archives - ouvertes . fr / hal - 02124411/document (visited on 12/17/2021).

[11] Q. Gendre and T. Kaneko, "Playing catan with cross-dimensional neural network," *Lecture notes in computer science*, pp. 580–592, Jan. 2020. DOI: 10.1007/978-3-030-63833-7_49.

[12] jdmonin, *Jsettlers2/readme.md at main · jdmonin/jsettlers2*, GitHub, 2021. [Online]. Available: https://github.com/jdmonin/JSettlers2/blob/main/Readme.md (visited on 04/17/2025).

[13] Arvind, *Akrishna77/cs7641-qsettlers*, GitHub, Apr. 2024. [Online]. Available: https : / / github . com / akrishna77/CS7641-QSettlers.

[14] B. Collazo, *Catanatron*, GitHub, Jan. 2024. [Online]. Available: https://github.com/bcollazo/catanatron.

[15] A. A. Mohd Nadhir Ab Wahab Samia Nefti-Meziani, "A comprehensive review of swarm optimization algorithms," *PLoS ONE*, vol. 10, 05 May 2005. DOI: https : / / doi . org / 10 . 1371 / journal . pone . 0122827. (visited on 04/18/2025).

[16] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015. DOI: 10.1038/nature14236. [Online]. Available: https://www.nature.com/articles/nature14236.

[17] D. Silver, A. Huang, C. J. Maddison, *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, Jan. 2016. DOI: 10.1038/nature16961. [Online]. Available: https://www.nature.com/articles/nature16961 (visited on 04/18/2025).

[18] J. Luo, W. Zhang, J. Su, and F. Xiang, "Hexagonal convolutional neural networks for hexagonal grids," *IEEE Access*, vol. 4, Oct. 2019. DOI: 10.1109/access.2019.2944766. (visited on 04/18/2025).